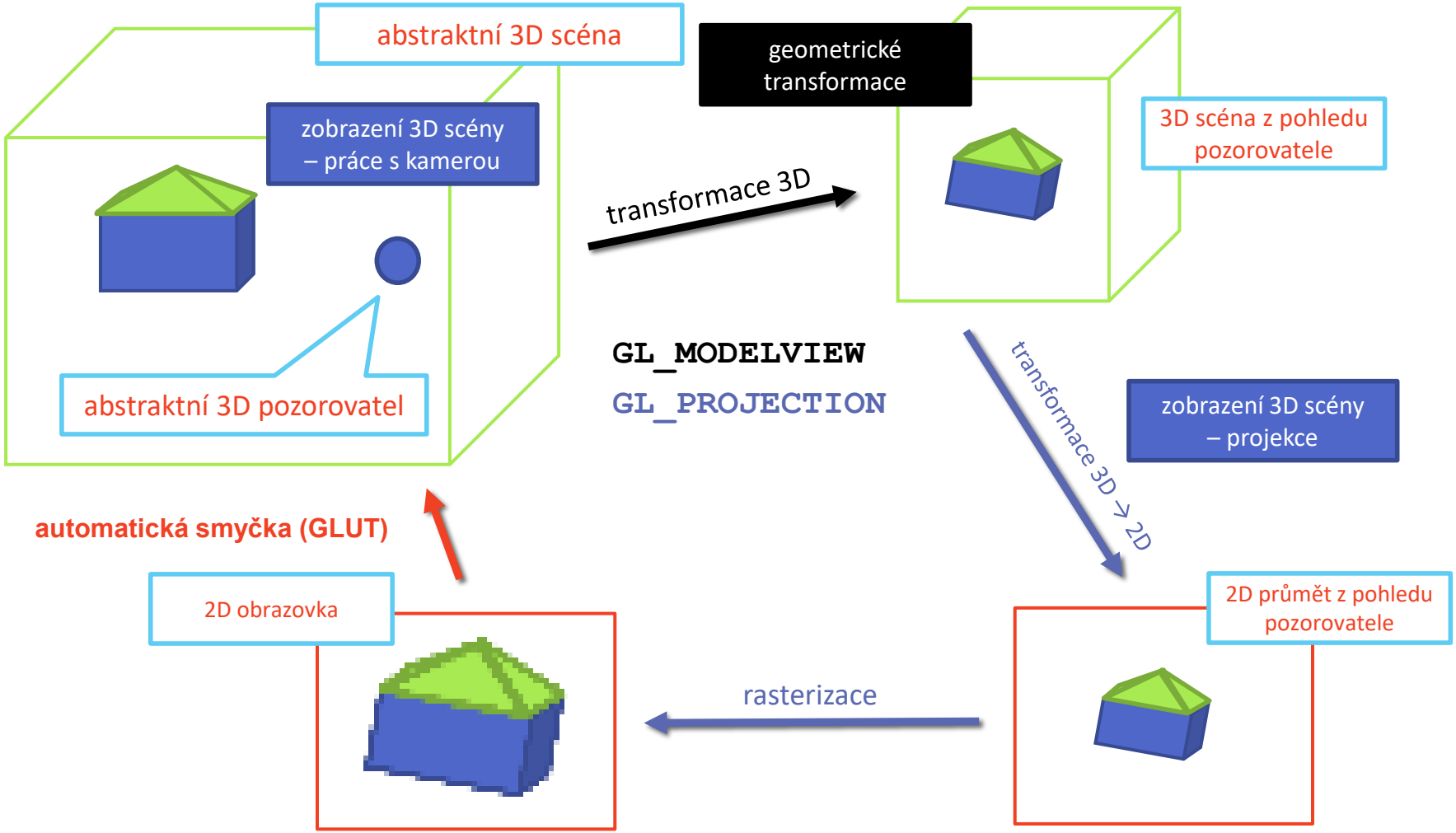
ZOBRAZENÍ 3D SCÉNY GEOMETRICKÉ TRANSFORMACE

Kurz: **Moderní počítačová grafika**

Lektor: Ing. Michal Švento

Náplň cvičení

1. Zobrazení 3D scény
   - Ortografická projekce
   - Perspektivní projekce
   - Práce s kamerou
2. Geometrické transformace
3. Transformace scény


abstraktní 3D scéna
zobrazení 3D scény – práce s kamerou
abstraktní 3D pozorovatel
geometrické transformace
transformace 3D
3D scéna z pohledu pozorovatele
GL_MODELVIEW
GL_PROJECTION
transformace 3D → 2D
zobrazení 3D scény – projekce
automatická smyčka (GLUT)
2D obrazovka
rasterizace
2D průmět z pohledu pozorovatele

Zobrazení 3D scény

- Nastavení módu pomocí matice:
  - `glMatrixMode(MATICE)`
- Na nastavenou matici se dále budou vztahovat jednotlivé operace (transformace).

- Matice:
  - **GL_MODELVIEW** = MODEL + VIEW = transformace objektu + pohledu (kamery)
  - **GL_PROJECTION** (ortogonální, perspektivní)
  - GL_TEXTURE
  - GL_COLOR

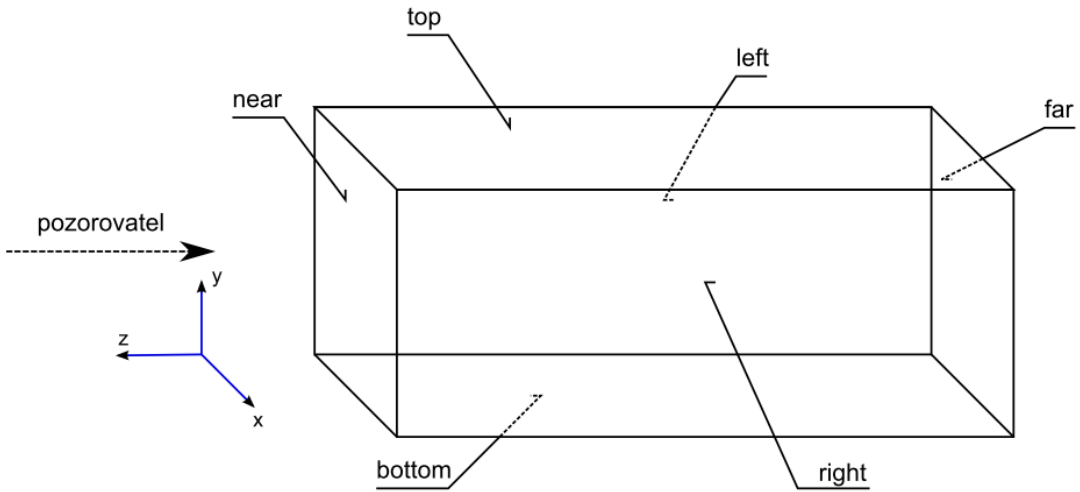
Zobrazení 3D scény

- Zobrazení 3D scény lze na výstupním 2D zařízení realizovat pomocí různých projekcí.
- Zobrazení souřadnic bodů z původních tří dimenzí do zobrazovaných dvou.
- Ortografické / perspektivní.

- Pohledový objem:
  - Pomocí průniku šesti ořezových rovin definujeme uzavřené těleso v prostoru abstraktních souřadnic.
  - Vše, co je mimo tento objem, se v zobrazení vypustí.

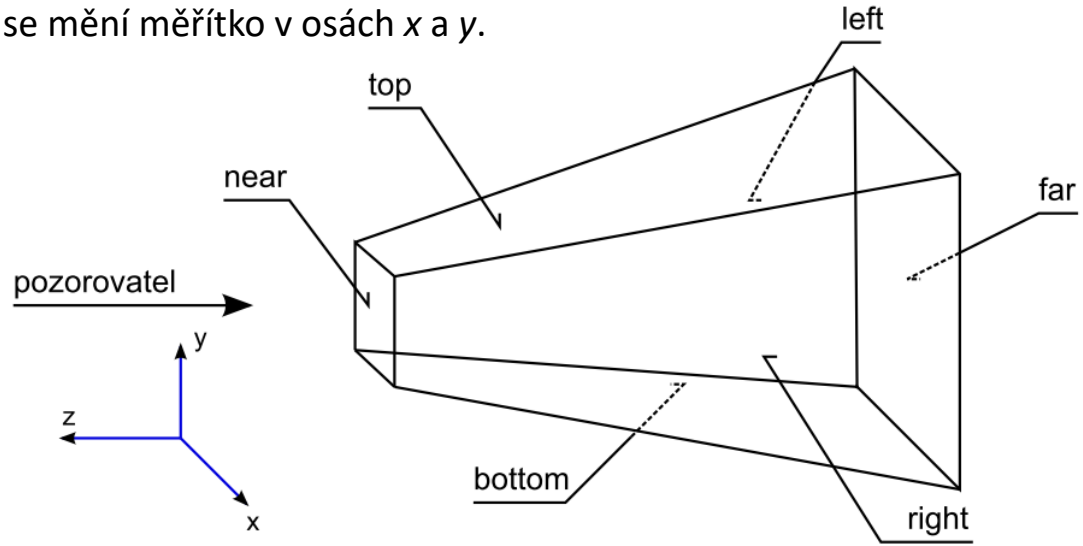
Ortografická projekce

- Z 3D bere v potaz pouze *x* a *y*, které beze změny převádí do 2D pohledu, souřadnice *z* se neprojeví.
  - `glMatrixMode(GL_PROJECTION);`
  - `glOrtho(double left, right, bottom, top, near, far);`
  - `gluOrtho2D(left, right, bottom, top);  // (near=-1, far=1)`

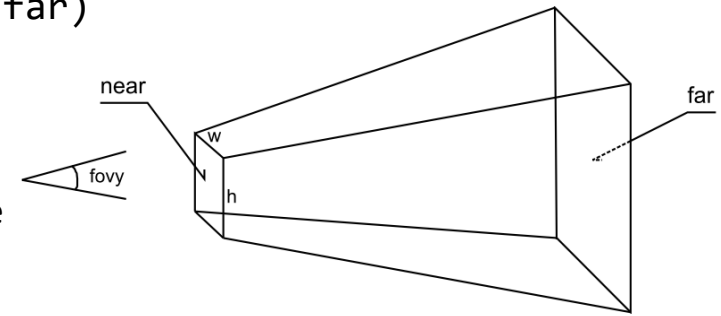


Perspektivní projekce

- S rostoucí vzdáleností podél osy *z* se mění měřítko v osách *x* a *y*.

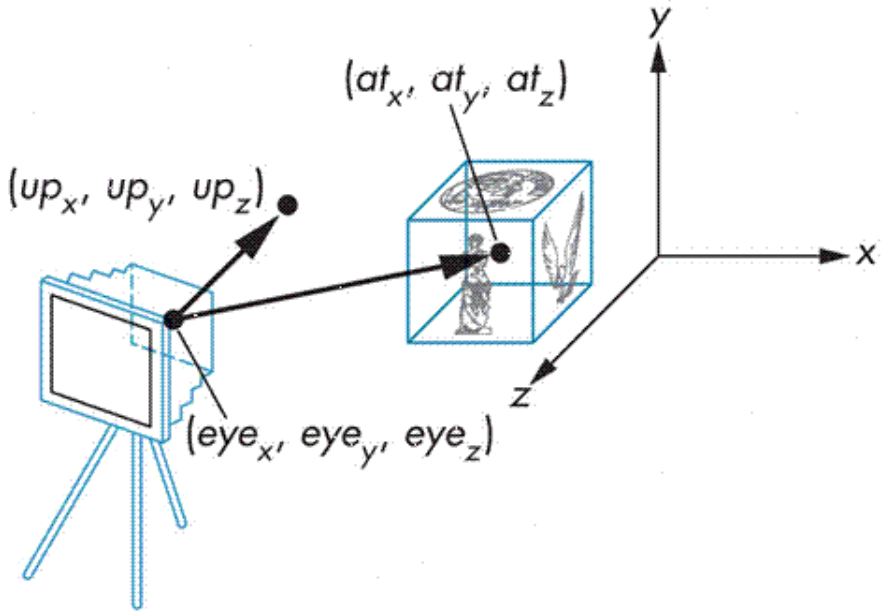


- `gluPerspective(double fov, aspect, near, far)`
  - `fov` – field of view, specifikace zorného úhlu
  - `aspect` – poměr w / h základny jehlanu
  - `near` – vzdálenost přední ořezové roviny od pozorovatele
  - `far` – vzdálenost zadní ořezové roviny od pozorovatele



Práce s kamerou

- `glMatrixMode(GL_MODELVIEW)`
- `gluLookAt(double eyeX, eyeY, eyeZ, centerX, centerY, centerZ, upX, upY, upZ)`
  - Kamera se nachází v bodě [eyeX, eyeY, eyeZ]
  - Sleduje bod [centerX, centerY, centerZ]
  - Je otočená tak, že vektor [upX, upY, upZ] směřuje nahoru



Úkol 1

1. Specifikujte ortogonální zobrazení s ořezovými rovinami [-20, 20, -20, 20, -30, 30].
2. Definujte pohled kamery z bodu [4, 4, 16] směrem na počátek souřadnic naší scény.
   - `gluLookAt()`
3. Přepněte zobrazení našeho útvaru na perspektivní s přednastavenými parametry funkce `gluPerspective()` a porovnejte výsledky.

onReshape() + onDisplay3D()

Geometrické transformace

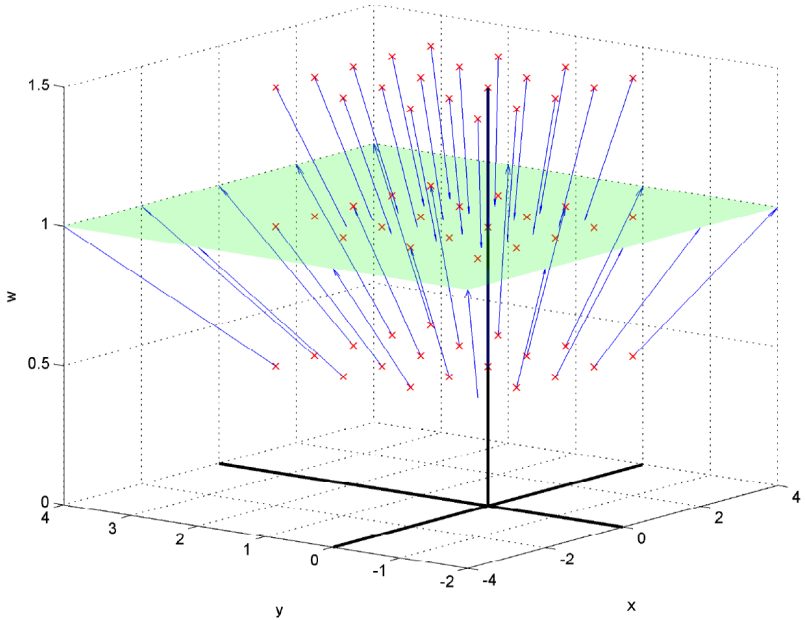
- Kartézské souřadnice: $[x \quad y \quad z]$
- Homogenní souřadnice: $[x \quad y \quad z \quad w]$
- Převod z kartézských na homogenní: $[x \quad y \quad z \quad 1]$
- Převod z homogenních na kartézské: $[x/w \quad y/w \quad z/w \quad 1]$

- Transformace scény odpovídá násobení bodů v homogenních souřadnicích čtvercovou maticí 4×4.
- Složení transformací odpovídá **násobení** jednotlivých transformačních **matic**.

$$\mathbf{T}_q = \mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \mathbf{T}_3 \ldots$$

$$\mathbf{T}_p = \mathbf{T}_2 \cdot \mathbf{T}_1 \cdot \mathbf{T}_3 \ldots$$

$$\mathbf{T}_q \neq \mathbf{T}_p$$



Transformace scény

- V OpenGL existují příkazy, které vytvoří matici příslušné transformace a tou je pak vynásobena aktuální matice:
  - Změna měřítka v jednotlivých osách
    - `glScalef(float x, y, z)`
  - Posun
    - `glTranslatef(float x, y, z)`
  - Rotace
    - `glRotatef(float angle, xaxis, yaxis, zaxis)`
  - Ručně definovaná matice transformace
    - `glMultMatrixd(double pole[16])`

Úkol 2

1. Do scény s použitou ortogonální projekcí vykreslete domeček pomocí funkce `vykresliDomecek()` a s domečkem proveďte následující operace:
   - Translace o 50 bodů v kladném směru osy *x* a o 50 bodů v kladném směru osy *y*.
   - Zmenšení měřítka na polovinu ve směru osy *x* i *y*.
   - Rotace o 15 stupňů po směru hodinových ručiček dle osy *z* (vektor [0, 0, 1]).
2. Bude se hodit…
   - `glOrtho(-double(w)/2, double(w)/2, -double(h)/2, double(h)/2, -1, 1);`

Otázky k zamyšlení

- Co se stane, pokud matici `GL_MODELVIEW` neinicializujete pomocí `glLoadIdentity()`?
- Zůstane výsledek výstupu stejný, když prohodíte jednotlivé transformace?

onReshape() + onDisplay2D()

Úkol 3

1. Implementujte automatickou rotaci domečku podle osy *z* o 5° za 15 ms.
2. Opatřete okno s rotujícím domem nápisem „Rotace domu“, který ovšem rotovat nebude.
   - Využijte funkci `bitmapText()`.
   - Použijte font `GLUT_BITMAP_HELVETICA_12`.
3. Aplikujte obdobnou animaci na 3D objekt ve window 1.

Bonus: změňte směr rotace domečku na opačný bez změny proměnné `angle`

onDisplay3D() + onDisplay2D() + onTimer()